CONFIDENTIAL CASE FILE

How They'd Actually Solve It

Not advice about coding — a transcript. Watch Patrick Jane and Mike Ross each sit down with the same stuck-on-logic problem and work it, live, in their own way.

Patrick Jane
CBI Consultant — Reads the Pattern First

Mike Ross
Associate, Pearson Specter — Builds the Brief First

01 The Setup

Forget tips. Here's the case, exactly as it was handed over: *"I know the syntax. I understand loops, conditionals, functions. But when I sit down to build something myself, without a tutorial next to me, I freeze. I don't know how to turn the problem into code."*

Two consultants, one file. Same problem, handed to both, cold. No prep. What follows is what actually happened when each of them sat down with it — not what they'd *tell you* to do, but what they *did*.

> THE PROBLEM ON THE TABLE
>
> Write a function that takes a string and returns the first character that does **not** repeat anywhere else in the string. If every character repeats, return `None`.
>
> Example: `"swiss"` → `"w"` (s repeats, w doesn't, i repeats).

02 Jane Sits Down With It

SCENE — JANE READS THE PROBLEM THE WAY HE READS A ROOM

He doesn't open an editor. He doesn't even open a notebook yet. He reads the problem twice, out loud, slower the second time.

> **JANE**
>
> *"'First character that doesn't repeat.' Everyone's going to rush at this by thinking about loops. That's the tutorial talking, not the problem. Let's ignore code entirely for a second. If I gave you the word 'swiss' on a piece of paper and a highlighter, and told you to find the answer with your eyes — no computer — what would you actually do?"*

He underlines each letter in `swiss` *by hand, marking how many times it shows up.*

```
s w i s s
1 1 1 2 3   ← how many times each letter appears total in the word
```

> **JANE**
>
> *"There. I didn't write a single line of code and I already have the answer — 'w' is the only one that shows up once. So the problem was never 'how do I loop through a string.' The problem was 'how do I count things, and then look for the first one whose count is one.' Two separate, much smaller problems. That's the trick — you were trying to solve 'find the non-repeating character' directly. Nobody can. You solve 'count everything,' then you solve 'find the first one that matches a condition.' Both of those, you already know how to do."*

JANE TURNS THE OBSERVATION INTO A PLAN

Only now does he write anything down — and it's still not code.

```
1. Count how many times each character appears in the string
2. Go through the string again, in order
3. The first character whose count is 1 — that's the answer
4. If nothing has a count of 1, return None
```

> **JANE**
>
> *"Notice step 2 says 'go through the string again, in order' — not 'look at the counts in order.' That's the detail people miss and then wonder why their code gives the wrong answer. The counts don't have an order. The original string does. If you skip working the tiny example by hand, you never notice that detail — you just guess, and the guess is usually wrong in exactly this kind of small way."*

NOW, AND ONLY NOW, CODE

```
def first_non_repeating(s):
    counts = {}
    for ch in s:
        counts[ch] = counts.get(ch, 0) + 1

    for ch in s:              # walking the ORIGINAL string, in order
        if counts[ch] == 1:
            return ch

    return None
```

> **JANE**
>
> *"I didn't think in code once until that last step. I thought in a highlighter and a piece of paper. The code is just me writing down a decision I'd already made. That's the whole secret, and it's not really a secret — it's just that everyone's in too much of a hurry to skip straight to typing."*

03 Mike Sits Down With It

SCENE — MIKE TREATS IT LIKE A CASE FILE, NOT A PUZZLE

Mike doesn't touch the problem the way Jane did. He opens a blank doc first — not to write code, to write an outline, the way he'd draft a brief before a filing.

> **MIKE**
>
> *"First thing I do with any case — I ask if I've seen this shape before. Not this exact problem. The shape of it. 'Something about how often things appear in a collection' — I've seen that shape a hundred times. Word frequency, duplicate detection, majority vote. Different surface, same underlying pattern: count things, then do something with the counts. That's precedent. I'm not solving from zero, I'm matching to something I've already built."*

MIKE DRAFTS THE BRIEF BEFORE TOUCHING A SINGLE LINE OF CODE

He writes exactly two things on the page: the inputs and outputs he's promising, and the plan — like a brief's summary section before the argument.

```
GIVEN: a string s
PROMISE: return the first character in s that appears exactly once,
         or None if there isn't one

PLAN:
  - Build a tally of every character (this is the "count things" pattern)
  - Walk the string in its original order, check each tally
  - Return on the first hit; return None if we finish with no hit
```

> **MIKE**
>
> *"That's the brief. Two loops, one dictionary. I know that shape cold because I've written it enough times that it's not a puzzle to me anymore, it's a template. That's the part people miss — you're not supposed to reinvent this from scratch every time. You're supposed to build a small library of shapes you've solved before, so next time you recognize it in three seconds instead of thirty minutes."*

HE BUILDS IT LIKE HE'S BUILDING AN ARGUMENT, ONE CLAUSE AT A TIME

```
def first_non_repeating(s):
    tally = {}                          # Exhibit 1: the count
    for ch in s:
        tally[ch] = tally.get(ch, 0) + 1

    for ch in s:                        # Exhibit 2: the order-respecting check
        if tally[ch] == 1:
            return ch                   # Verdict: found it

    return None                         # Verdict: no such character exists
```

> **MIKE**
>
> *"Every line has a job, and I can tell you what it is before I write it, because I wrote the brief first. If you can't explain what a line of code is doing before you type it, you're not ready to type it yet — go back and finish the brief. That's not me being precious about process. That's the difference between debugging for two minutes and debugging for two hours."*

> **MIKE**
>
> *"And here's the part that actually matters for you, long term — I didn't know this pattern the first time either. I got handed a case, I didn't recognize the shape, and it took me forever. The only reason I see it instantly now is I've solved close cousins of it a dozen times since. You build the library by doing the reps, not by reading about them."*

04 Where They Actually Agree

Jane reads the transcript of Mike's approach. Mike reads Jane's.

> **JANE**
>
> *"He calls it a 'brief,' I call it 'working it by hand.' Different words, same move — neither of us opened an editor first. We both refused to think in code until we'd already solved the problem in plain terms. That's not a coincidence."*

> **MIKE**
>
> *"He's right. My 'pattern library' is really just his 'hand-worked example,' done enough times that I don't need to work it by hand anymore — I recognize it on sight. Which means his method is how you build my method. You don't start with the shortcut. You earn it."*

STEP	JANE'S VERSION	MIKE'S VERSION
1	Read the problem twice, out loud, before anything else	Ask "have I seen this shape before?" — name the pattern
2	Solve one tiny example by hand, no code	Write the brief — inputs, promise, plan — no code
3	Notice the small details the hand-example reveals	Break the plan into 2–3 named sub-steps ("exhibits")
4	Only now, translate the hand-solution into code	Only now, translate the brief into code, clause by clause
5	Trust nothing you didn't just personally work out	Explain every line's job before you write it — or don't write it yet

> **THE ONE RULE BOTH OF THEM ACTUALLY USED**
>
> Neither of them started with code. Both of them refused to type a single line until the problem had already been solved in plain language, on paper or in a doc. The code, for both of them, was the *last* step, not the first — a translation of a decision already made, not the place where the deciding happens.

05 Try It Yourself — Same Method, Blank Case

Next time you're stuck, don't open the editor. Open a blank page and run this, in order, the way you just watched them do it:

1. **Read it twice, out loud.** If you can't restate the problem in your own words, you're not ready to solve it yet.
2. **Work one tiny example by hand.** Real values, on paper. Not "in your head" — actually written down, the way Jane marked up "swiss."
3. **Name the shape, if you recognize it.** Counting things? Comparing pairs? Searching for a target? Even a rough guess narrows the problem.
4. **Write the plan in plain English — 3 to 6 steps, no syntax.** This is the brief. Skipping it is the single most common reason people freeze.
5. **Translate the plan into code, one line per step.** You're not thinking anymore — you're translating a decision you already made.
6. **Trace it against your hand example.** If it doesn't match, you now know exactly which line to look at — not the whole file, one line.

Case Closed

"You weren't missing knowledge. You were missing a middle step, and you were in too much of a hurry to notice it was missing. Slow down for thirty seconds at the start of every problem. That's the whole fix."

— PATRICK JANE

"And do it on purpose, every time, even the easy ones — especially the easy ones. That's how the pattern library gets built. One day you'll sit down with a problem and just... know the shape of it. That's not talent. That's reps."

— MIKE ROSS